第1回：行列・座標変換のC#実装例

詳細解説 と 宿題の問題・解答 の計算を、手実装と `System.Numerics` で比較する。列ベクトル方式の `double` の式と、.NETの行ベクトル方式のAPIを通じて、点・方向・基底・合成順序を確かめる。

ファイルと読む順序

ファイル	数学的な役割
Program.cs	6題材の入力、API呼び出し、期待値、自動チェック
ColumnMath.cs	列ベクトル方式の掛け算と.NETの配置への変換

`Program.cs` の各題材から対応する `ColumnMath` の式を読む。手実装はこの教材の2×2・3×3・4×4に絞り、配列を `[行, 列]` で参照する。2Dの3×3は最下行が `(0,0,1)` のアフィン変換だけを扱う。透視投影や一般の行列ライブラリは対象にしない。

各題材の冒頭コメントで検証意図と対応する解説・宿題を確認し、`ColumnMath` の計算式を読んでから、呼び出し元の.NET APIと `Check` の期待値へ戻る。日本語XMLコメントには入力・出力・前提を記載している。

手実装と.NET APIは、基底の変換先、点と方向の `w`、転置後の移動成分、合成順序、`double` から `float` への精度の変化に注目して比較する。

実行順・入力・期待結果

X軸は右、Y軸は上、正の2D回転は反時計回り。3Dは右手系で、Z軸の正回転は+Z側から原点を見た反時計回りとする。回転の中心は原点。

実行順	対応する教材	主な入力と期待結果
1. 行列×ベクトルと基底	解説②	本文の2×2行列で `(2,1) → (5,4)`。X基底は `(2,1)`、Y基底は `(1,2)` へ移る
2. 2D回転	解説③・基本問題1	90° で `(2,1) → (-1,2)`、基底は `(0,1)` と `(-1,0)`。30° では約 `(1.23205081,1.8660254)`
3. 同次座標と平行移動	解説④・基本問題2	移動量 `(3,-2)`。点 `(2,1,1) → (5,-1,1)`、方向 `(2,1,0) → (2,1,0)`
4. 3Dのローカル→ワールド	解説⑤・発展問題	Z軸90° 回転後に `(10,0,3)` 移動。点 `(2,1,0,1) → (9,2,3,1)`、方向 `(2,1,0,0) → (-1,2,0,0)`
5. 合成の順序	解説⑥・基本問題3	`(1,0)` の回転→X+3移動は `(0,1) → (3,1)`、移動→回転は `(4,0) → (0,4)`
6. .NETの転置配置と精度	解説⑥・補足	非対称な係数の行列で配置を検証し、`(2,1) → (8,8)`。doubleとfloatの三角関数の残差も表示

出力例（抜粋）：

```
1. 行列×ベクトルと基底（解説②）
  [OK] Ap: (5, 4)
  [OK] Ae1 = 第1列: (2, 1)
  [OK] Ae2 = 第2列: (1, 2)
...
  [OK] 90度の点: (-1, 2)
  [OK] 30度の点: (1.23205081, 1.8660254)
...
  [OK] 点 w=1: (5, -1, 1)
  [OK] 方向 w=0: (2, 1, 0)
...
  [OK] ワールドの点: (9, 2, 3, 1)
  [OK] ワールドの方向: (-1, 2, 0, 0)
...
  [OK] 回転→移動: (3, 1)
  [OK] 移動→回転: (0, 4)
...
すべての 43 チェックに成功しました。
```

数式とコードの対応

掛け算・回転・同次座標

`Multiply2` は行ごとの掛け算を直接書いている。例えばX成分の `m[0, 0] * p.X + m[0, 1] * p.Y` は `2*2 + 1*1 = 5` に対応する。第1列と第2列は、それぞれ標準基底の変換先になる。入力が $p = 2e_1 + e_2$ なら、出力は $2Ae_1 + Ae_2$ となることも検証する。

`Rotation2` はラジアンで受け取った角度から次の行列を作る。`Program.cs` では $\theta = 30°$ を使い、度からラジアンへの変換も示す。0° ・90° ・180° ・−90° ・360° でも同じ式を検証する。

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

第1列は回転後のX基底、第2列は回転後のY基底。30° の期待値は $\cos 30° = \sqrt{3}/2$ と $\sin 30° = 1/2$ から別に求め、回転で長さが変わらないことも確認する。

`Transform2`・`Transform3` は同次成分も省略せず計算する。例えば2D平行移動は $x' = x + 3w$ 、 $y' = y - 2w$ なので、点の $w = 1$ では移動を足し、方向の $w = 0$ では足さない。原点が移動することも確認できる。

Matrix3x2は3×3同次行列の省略表現

本文の列ベクトル方式で、2Dアフィン変換を次のように書く。

$$M_{col} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix}.$$

.NETの行ベクトル方式で同じ変換を表すには転置した配置にする。

$$M_{row} = M_{col}^{\mathsf{T}} = \begin{bmatrix} a & c & 0 \\ b & d & 0 \\ t_x & t_y & 1 \end{bmatrix}.$$

`Matrix3x2` はこの固定された最後の列 `(0,0,1)` を省いた6成分を保持する。コンストラクターの引数は `new Matrix3x2(a, c, b, d, tx, ty)` の順になり、`ToRow3x2` がこの対応を実装している。本文の上2行をそのまま渡す配置にはしない。

変換	移動量を置く.NETのフィールド	点の変換	方向の変換
2D	`Matrix3x2.M31, M32`	`Vector2.Transform`	`Vector2.TransformNormal`
3D	`Matrix4x4.M41, M42, M43`	`Vector3.Transform`	`Vector3.TransformNormal`

`ToRow4x4` も本文の4×4を転置する。`TransformNormal` は移動を足さず線形部分を掛けるので、この教材では方向の変換に使う。一般の法線変換は扱わない。

合成順序も対応させる

列ベクトル方式で回転後に移動すると、$p' = T_{col} R_{col} p$。`Multiply3` で左の行と右の列の積を足し、宿題の TR と RT の全9成分、逐次適用と一括適用の一致を確認する。

同じ回転→移動を行ベクトル方式にすると、$p'_{row} = p_{row} R_{row} T_{row}$ となる。コードでも次の順に掛ける。

```
var placement = Matrix4x4.CreateRotationZ(MathF.PI / 2)
              * Matrix4x4.CreateTranslation(10, 0, 3);
var world = Vector3.Transform(new Vector3(2, 1, 0), placement);
```

これはベクトルを掛ける側の規約の違いであり、メモリの格納順序を説明しているわけではない。規約は Matrix3x2 と Matrix4x4 の公式資料のRemarksでも確認できる。

精度と自動チェック

手実装は `double`、`System.Numerics` の各成分は `float`。π も有限精度なので、数学では0になる計算にも微小な残差が出る。

```
Math.Cos(Math.PI / 2)   = 6.123233995736766E-17
MathF.Cos(MathF.PI / 2) = -4.37113883E-08
```

数値の末尾は実行環境で異なる場合がある。APIには特定の角度で値を補正する実装もあるため、すべての90° 回転APIが必ず同じ残差を返すという意味ではない。

`Check` は各成分の絶対誤差を比較する。この教材の小さな固定入力では、double同士に `1e-12`、floatを含む比較に `1e-5` を使う。厳密な等号や、表示時に丸めた値で判定しない。NaN・Infinityも失敗とし、期待値を外れた箇所のラベル・実値・期待値・許容誤差を表示する。入力の桁数や大きさを変える場合は許容誤差も見直す。

検証範囲

6題材・43件の自動チェックで、基底の変換、回転、点と方向、ローカル→ワールド、合成順序、転置配置を固定値と照合する。手実装と.NET APIの比較では、`double` と `float` の精度を分ける。検証はこの教材の固定入力に対するもので、任意の行列や透視投影への対応を保証しない。

資料の元データ（GitHubリポジトリ）： https://github.com/mkunori/geometry-study-group